The Seoul Report on Systems Research

LORENZO ALVISI*, HAIBO CHEN, PETER CHEN, BYUNG-GON CHUN, DONG DU, GERNOT HEISER, FRANS KAASHOEK, ANA KLIMOVIC, CHRISTOS KOZYRAKIS, BUTLER LAMPSON, JEANNA MATTHEWS, JEFF MOGUL, KYOUNGSOO PARK, DAVID PATTERSON, ROBBERT VAN RENESSE, TIMOTHY ROSCOE, DIMITRIOS SKARLATOS, ION STOICA, YUBIN XIA, HARRY XU, JUNFENG YANG, DING YUAN, and MINGXING ZHANG

Systems researchers met at SOSP 2025 to reflect on our community's impact and to examine the research challenges created by AI, specialized hardware, large-scale infrastructure, and the growing difficulty of building reliable systems.

Additional Key Words and Phrases: systems research, operating systems, artificial intelligence, reliability, hardware–software co-design

ACM Reference Format:
Lorenzo Alvisi, Haibo Chen, Peter Chen, Byung-Gon Chun, Dong Du, Gernot Heiser, Frans Kaashoek, Ana Klimovic, Christos Kozyrakis, Butler Lampson, Jeanna Matthews, Jeff Mogul, KyoungSoo Park, David Patterson, Robbert van Renesse, Timothy Roscoe, Dimitrios Skarlatos, Ion Stoica, Yubin Xia, Harry Xu, Junfeng Yang, Ding Yuan, and Mingxing Zhang. 2026. The Seoul Report on Systems Research. 1, 1 (September 2026), 8 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

Systems research has repeatedly turned difficult technologies into dependable foundations. Operating systems made hardware usable across applications. Distributed systems made many machines serve as one. Virtualization, storage, networking, and cloud platforms carried these ideas into infrastructure used at global scale. The influence also runs in the other direction: deployment experience has exposed new failure modes, forced abstractions to evolve, and supplied the problems that keep our field relevant.

Isolation, predictable resource management, and reliable recovery remain difficult to achieve together in practical systems. These longstanding challenges are evolving alongside changes in the hardware and software landscape. General-purpose performance scaling has slowed. Domain-specific accelerators and new memory technologies are fragmenting the hardware substrate. AI is becoming both a major systems workload and a tool for producing systems. Meanwhile, long software supply chains, wide cloud interfaces, and tightly coupled services make it harder to know what a system will do, why it failed, or whom to trust.

In October 2025, the authors of this report met in Seoul, South Korea, for the first ACM SIGOPS Strategic Workshop, held with SOSP 2025. The meeting brought together researchers and practitioners from operating systems, distributed systems, cloud infrastructure, architecture, reliability, and AI. It followed the spirit of the 2015 SOSP History Day and the database community's periodic research reports. We looked backward for durable lessons and forward toward problems that may shape the next five to ten years.

This report summarizes recurring workshop themes and possible research directions drawn from those discussions. It is intentionally selective. It does not survey the full breadth of systems research, nor does it claim that every participant preferred the same solution.

*Organizers and participants, listed alphabetically by last name.

1 KEY TAKE-AWAYS

- **Building dependable operating systems remains an open challenge.** Microkernels and formal verification provide foundations for isolation and correctness. Building usable systems on these foundations raises open questions about service composition, resource management, and performance.
- **AI expands the scope of systems research.** Lower prototyping costs can make more ambitious systems feasible, while bringing problem formulation and evaluation into sharper focus. One option is to assess benchmarks, simulators, and specifications for AI-built systems as research contributions in their own right.
- **Learned components and agents call for new system tools.** Their behavior depends on data and model parameters as well as code. This creates opportunities for new tools spanning data preparation, training, testing, verification, runtime monitoring, etc. These tools must contend with both accidental faults and deliberate manipulation.
- **Reliability is a property of the whole system.** From device drivers and file systems to distributed services, reliability depends on how components interact and how failures are contained. Trustworthy components and reliable protocols do not automatically compose into a dependable system. This raises questions about deployment, performance, feedback loops, diagnosis, and persistent hardware faults.
- **Co-design and stable interfaces offer complementary benefits.** Specialization creates opportunities for coordination across models, software, hardware, memory, and operations. Narrow and declarative interfaces support portability, evolution, and verification.
- **Our community's knowledge and practices are research infrastructure.** The discussions raised questions about preserving engineering judgment and turning tacit craft into transferable principles in an AI-assisted world. Venues could experiment with a synthesis track for surveys, reference models, negative results, and abstractions validated across systems. Participation, responsibility, and face-to-face exchange also emerged as themes in considering the field's progress.

2 WHAT HAS CHANGED FOR SYSTEMS RESEARCH?

Three changes cut across nearly every workshop session. The first is a new hardware balance. The end of Dennard scaling moved performance growth away from faster general-purpose cores and toward parallelism and domain-specific architectures. AI has accelerated that movement, but arithmetic throughput alone gives an incomplete picture of the machine. Memory bandwidth and capacity, data movement, energy, packaging, and the placement of data across heterogeneous memories increasingly determine performance and cost. For the OS and runtime, scheduling increasingly involves where computation and state reside, as well as when work runs. Provisioned power at the rack and facility level can also limit the capacity available to AI deployments. Within these limits, reducing energy per unit of useful work can increase sustainable throughput. CPUs, GPUs, NPUs, and memory technologies such as HBM, DRAM, and CXL-attached memory have different performance–power tradeoffs, making placement and scheduling part of power management.

The second change is the dual role of AI. AI workloads bring probabilistic quality, enormous synchronous jobs, rapidly changing models, and a growing collection of serving and agent components. At the same time, AI tools now inspect and rewrite system implementations, going beyond tuning a black box. An agent proposes a design, evaluates it, and iterates. This loop brings problem formulation, measurement, and verification into focus. An underspecified task or an evaluator that rewards the wrong behavior risks directing optimization toward the wrong objective.

The third change is the scale of composition. Modern services combine hardware, firmware, compilers, kernels, libraries, distributed protocols, deployment systems, provider APIs, and learned components. Each layer may be reasonable in isolation while the whole remains fragile. A local loss of capacity may become a performance outage. A temporary overload may trigger a degraded state that persists after the overload ends. A rollout or mistaken diagnosis may do more damage than the initiating fault. These failures highlight the close relationship between design, operations, and maintenance. Experience in operation reveals behavior that is difficult to anticipate during design.

These changes pull systems research in two directions. Efficiency pushes us down the stack, toward model internals, memory hierarchies, specialized devices, and deployment details. Longevity pushes us toward simple mechanisms, declarative descriptions, and stable interfaces that allow pieces to evolve independently.

3 RESEARCH CHALLENGES

The changed landscape presents many opportunities. We organize the workshop's discussion into four areas in which systems expertise is especially relevant: (1) co-designing systems and AI, (2) achieving reliability across the full system, (3) rebuilding stable interfaces and OS foundations, and (4) making systems knowledge transferable.

3.1 Co-Designing Systems and AI

The workshop discussion highlighted three roles for AI. It is a workload supported by systems infrastructure, a component inside systems, and an instrument for systems research. The roles share mechanisms but not success criteria. A serving system couples output quality with latency and throughput. A learned component inside an OS raises familiar questions about testing, monitoring, and repair. Its logic, however, is encoded in model state rather than source statements. For an agent that rewrites code, success depends on preserving the intended system behavior; a convenient benchmark may capture only part of that behavior. Conflating the roles risks building a fast tool that solves the wrong problem.

More ambitious systems prototypes. AI-assisted development can substantially lower the cost of prototyping complete systems. This gives academic groups room to explore more ambitious designs, connect ideas across layers, and compare alternatives that previously required too much engineering effort. An OS idea, for example, may become easier to study together with the runtime and application that exercise it. As implementation becomes less costly, more of the research challenge lies in formulating a worthwhile problem, specifying the intended behavior, and designing an evaluation that exposes meaningful tradeoffs.

Evaluators as research artifacts. As AI tools move from parameter tuning to white-box construction, evaluation becomes part of the design. Evaluators face a challenge: how to assess correctness, performance, resource cost, robustness, and operational constraints together. A single score risks obscuring differences among these goals. Aggregate measures such as overall accuracy can hide behavior on rare or unforeseen inputs, where failures can have serious consequences. An apparent improvement in the measured result also raises the question of whether the intended behavior has been preserved. Possible approaches include diverse and held-out workloads, better simulators, and specifications that help explain why a proposed change works and where it fails.

Tools for learned components and agents. Learned components and agents introduce tooling challenges from data preparation and training through deployment and maintenance. Traditional tools offer a starting point, but learned behavior is encoded in opaque model parameters as well as code. For example, high statement coverage in inference code says little about which learned behaviors a test has exercised. Research opportunities include testing with model-aware coverage metrics and verification methods suited to numerical computation. After deployment, monitors could detect

shifts in input data or behavior, while repair tools could adapt models or remove harmful training-data influence. How these tools fit together across training, fine-tuning, and inference remains an open question.

Accuracy-aware infrastructure. AI services introduce tradeoffs between system performance and output quality, beyond a binary notion of correctness. Quantization, caching, speculative execution, and continuous batching offer ways to improve efficiency. Yet numerical choices and even batch composition may change results. Explicit quality objectives could help production systems balance these choices with latency, throughput, availability, and cost. Slow or failed accelerators and loss of a cloud provider add reliability challenges to the same stack. Agentic workloads add persistent state, tools, orchestration, and computer use. The basic abstractions that tie these elements together are still evolving.

Model-facing OS abstractions. Treating a model as an ordinary process preserves compatibility but may hide opportunities for resource management and protection. Replacing the operating system with a model risks losing useful mechanisms. An evolutionary approach offers another direction. Possibilities include declarative interfaces suited to an agent's goals and explicit management of KV-cache and other model state. Scheduling across CPUs, GPUs, NPUs, memories, and remote services is another area for exploration. Such integration also widens the security boundary. An agent acting with a user's authority raises questions about constrained delegation, privacy, and auditability. These questions concern both machine actions and access to models and data.

Efficiency across the workflow. The gap between AI demand and conventional hardware improvement strengthens the case for co-design. Considering efficiency constraints early could help guide model and algorithm design. System software connects new accelerators to applications, while automated exploration offers ways to compare designs across the stack. The relevant cost often extends beyond the GPU. CPU work, DRAM capacity, storage, data movement, and tools invoked by agents may dominate parts of a workflow. Power illustrates the need for coordination across these resources: a device-level optimization may shift consumption elsewhere or leave a shared power budget underused. OS and runtime mechanisms could coordinate fine-grained power controls and workload placement with rack- and facility-level limits, balancing energy efficiency with application performance. Research that reaches production quickly illustrates the opportunity for impact. It also highlights a risk: optimizing a single kernel may shift cost elsewhere in the workflow.

Large clusters are only one setting for exploring these questions. Simulation, emulation, focused testbeds, and studies of inference or fine-tuning offer other ways to examine architectural and systems problems. Academia also has opportunities to explore alternatives to the dominant hardware and software ecosystem. Today's stack is deeply optimized around dominant models and operations. This hardware lottery may make alternatives appear impractical before their software exists. Intermediate representations, compilers, runtimes, and evaluation tools could help new hardware ideas run meaningful workloads without reproducing an entire ecosystem.

3.2 Reliability Across the Full System

Abstraction allows us to use a component without understanding its implementation, but not without depending on it. Trust passes transitively through libraries, compilers, operating systems, hardware, manufacturing, models, training code, and the tools that produced them. An explicit trust model identifies which components a system assumes to be trustworthy and which it does not. Risk analysis complements that model by examining the consequences of each trust assumption. How likely is the assumption to fail, and how much control would that failure grant? Least privilege, isolation, focused verification, diversity, and redundancy offer ways to limit control or avoid a single point of trust.

Trustworthy OS foundations. OS kernels remain central to open questions about security, fault containment, and resource management. Microkernel designs keep privileged mechanisms small and place services such as device drivers and file systems in separate user-level components. Formally verified microkernels such as seL4 show how rigorous proofs can establish kernel correctness and isolation under explicit assumptions. A possible continuing challenge is to carry these guarantees through the services and policies that make an operating system usable. How can independently developed services be composed so that their interactions preserve system-wide properties? These questions connect kernel architecture and formal reasoning to the construction of dependable embedded and general-purpose systems.

Adversarial threats to learned components and agents. Trustworthiness also depends on resilience to deliberate manipulation. Attackers may poison training data, craft inputs that induce incorrect outputs, or use prompt injection and jailbreaks to redirect model behavior. For agents with access to tools and user privileges, such manipulation may affect actions as well as answers. The model supply chain adds another concern: unclear provenance complicates trust decisions, and a compromised model-loading process may execute malicious code. These threats raise questions about what an adversary controls, how attacks are detected, and how their effects are contained. Adversarial testing and runtime monitoring could complement isolation and constrained delegation in exploring these questions.

Availability beyond protocols. Our field has reliable protocols, yet services built from them still fail. Availability includes completing useful work at acceptable performance, not merely returning a correct answer eventually. A fault-tolerant component that loses capacity can trigger queues and timeouts in its callers. A single slow participant can stall a synchronous AI job. Configuration changes and staged rollouts alter the system while it is running. A plausible but wrong diagnosis may prolong an outage. Alongside protocol refinement, deployment, observability, diagnosis, and repair offer research opportunities for improving availability. Models, experiments, and tools for these activities could help connect protocol guarantees to service behavior.

Failures created by composition. Metastable failures show why component-wise reasoning is insufficient. A temporary trigger can push a service into a degraded state that persists after the trigger disappears. Retry amplification is one example. More generally, individually stable components may change one another's operating environments and enter a cycle of mutually destabilizing interactions. This cycle corresponds to a design fault, and it is often inevitable. Fault tolerance, however, is possible by augmenting the composition with a scheduler that prioritizes stabilizing interactions. Compositional models could help explain when local control loops preserve global stability.

Practical beachheads of correctness. Formal verification can establish that an implementation satisfies a precise specification under explicit assumptions. Extending these proofs to complete systems and maintaining them as code evolves remain difficult. Gradual verification offers a way to work with production code without first proving the entire system. It starts with the state, invariants, functions, and code paths that matter most, then finds a focused bug or proves a focused property. The verified region can grow as the system changes. Risk analysis could help prioritize proof effort toward components whose failure grants the greatest control or causes harm that is difficult to reverse.

Reliability above faulty hardware. Persistent and silent hardware faults weaken the assumption that restart returns execution to a sound base. A mercurial core may reproduce an error after checkpoint recovery, while silent corruption can escape conventional failure detectors. Software-defined checking and redundancy offer ways to use otherwise idle resources to detect or mask these faults. Two questions follow: how much coverage is possible at acceptable overhead, and how does hardware evidence support a service's end-to-end correctness claims?

3.3 Stable Interfaces and OS Foundations

Specialization often rewards vertical integration. Tuning an application, runtime, cloud service, and accelerator together offers immediate gains. Over time, however, tight coupling may obstruct portability, independent innovation, and verification. Modern cloud platforms illustrate this tension. Instead of a narrow waist with many implementations below and applications above, they expose a wide and changing collection of provider-specific APIs. AI-generated software adds another dimension to the problem. Each additional interface expands the behavior that the generator and evaluator have to account for.

Narrow waists through experience. Earlier OS, network, and data management stacks illustrate the value of stable interfaces. By exposing a small common mechanism, these interfaces supported independent evolution. Similar waists are an appealing possibility for the cloud, although it may be difficult to define them before gaining experience with enough systems. Narrow waists often emerge after experimentation reveals what is truly common. Research could contribute by comparing real services and identifying repeated mechanisms. Candidate interfaces could then be tested for portability and their effects on important optimizations.

Declarative interaction. An imperative sequence forces the caller to coordinate intermediate state and often encodes accidental details of one implementation. A declarative request states the desired outcome and leaves room for the system to optimize, recover, and choose among heterogeneous resources. This idea connects model-facing interfaces, cloud portability, configuration, and operations. It also raises questions about verification and explanation. How precisely does the declaration describe the desired outcome, and what evidence explains how the system achieved it?

Revisiting OS design. OS design continues to face the challenge of combining strong isolation, efficient resource sharing, and usable abstractions. A small verified kernel offers strong low-level guarantees, while leaving system builders to construct services such as device drivers, file systems, and memory management above primitive interfaces. For embedded and general-purpose systems, how can these services be organized into a usable whole whose security and reliability can be understood and verified? Radical simplicity, fine-grained modules, and separation of concerns offer one direction. Sequential event-driven logic within modules and replaceable use-case-specific policies are related possibilities. The appeal is practical: mechanisms that are easier to understand, verify, maintain, and adapt while retaining performance.

Co-design and stable interfaces introduce a tradeoff in how much information crosses a boundary. Hiding distinctions among accelerators and memories may hinder efficient placement. Exposing every device detail risks moving interface complexity upward. What is the smallest amount of information that supports good placement, protection, and diagnosis? Classic systems design advice offers useful starting points: clear goals, smaller subproblems, and incremental evolution. A good-enough mechanism may also be preferable when greater generality makes a system too difficult to understand or operate.

3.4 From Systems Craft to Transferable Knowledge

Much of what makes a strong systems researcher effective is tacit knowledge. It includes choosing boundaries, interpreting anomalous measurements, simplifying designs, and debugging across layers. It also includes recognizing when an optimization is likely to fail in operation. This knowledge moves through apprenticeship and is repeatedly rediscovered in individual projects. Our field produces many systems, but it is less systematic about extracting the

principles that survive them. This makes it harder to organize related work and maintain useful teaching material. Insights may remain tied to products or hardware that disappear before the next generation can use them.

Recognizing synthesis as a research contribution offers one way to make this knowledge more transferable. Surveys, reference models, shared artifacts, negative results, and abstractions validated across multiple systems can reveal where results agree and where design spaces remain unexplored. This is not an argument to stop building. Concrete systems are how we discover behavior and test ideas. The opportunity is to make the reusable knowledge extracted from them an explicit research output.

AI adds an educational dimension to this discussion. Code generation offers productivity gains, but generated code alone does not explain why a boundary is appropriate or how to recognize a plausible failure. One educational approach is to pair these tools with practice in problem formulation, evaluation, and explanation. The limits of automated verification also make inspecting and challenging generated results a valuable systems skill. A codified body of systems knowledge could support the tools themselves. Specifications, design alternatives, and failure patterns may prove more durable than examples drawn from a particular codebase.

4 COMMUNITY

Technical priorities alone do not determine the community's future. Participation and the incentives created by our venues influence which problems receive attention. Education shapes which lessons survive, while the consequences of deployed systems raise questions about responsibility.

Conferences as places for interaction. Publication and community building are related but different functions. Paper review serves a different purpose from meeting in person. Time for practitioners, tutorials, focused discussions, and early ideas could enrich the conference experience. So could exchange across seniority and research areas. Workshops and experimental tracks offer settings to try new formats and assess their outcomes. A larger publication pipeline does not necessarily lead to a stronger community.

Participation and representation. The geographic center of systems research is broadening. Workshop data compared 2016–2020 with 2021–2025. The average number of papers per conference from Asian institutions rose from about 6 to 15.2. The numbers of active faculty and contributing institutions also grew sharply. Yet program committee representation remained about 11.3%, below a publication share approaching 30%. Although these measures deserve continued scrutiny, they raise a question about representation: how might governance and service better reflect the community's expanding research base?

Judgment and sustainability in education. How can we help students use AI tools while retaining responsibility for explaining and verifying the results? One approach is to give problem statements, invariants, evaluation, and operational reasoning attention alongside implementation. The fast replacement cycle of AI hardware creates another systems opportunity: aging accelerators may still serve other workloads, extending their useful life and reducing embodied-carbon cost. This suggests a broader view of resource efficiency that considers a machine's lifetime alongside device utilization.

Responsibility for system effects. System design has consequences beyond technical performance. System interfaces and defaults distribute power and determine what can be observed. They also influence whether a harmful decision can be challenged. Observability, provenance, explanation, access, redress, redundancy, and independent sources of truth offer possible technical safeguards. Their suitability depends on both the likelihood and severity of harm, especially when recovery is impossible.

5 CONCLUSION

Systems research is entering a period in which its workload, substrate, and design process are changing together. AI creates new infrastructure demands and new ways to explore designs. Specialization creates opportunities for hardware software co-design. Scale exposes failures that component-level models may miss. These changes give renewed relevance to work close to implementation. They also highlight the value of simple mechanisms, stable interfaces, and knowledge that survives a particular artifact.

Our community brings a long tradition of developing abstractions, characterizing tradeoffs, and building mechanisms. Measuring whole systems and revising designs in light of experience are part of that tradition. Over the next five to ten years, we may see those habits increasingly applied across models, hardware, software, operations, and human institutions. Applying these habits more broadly could help make changing technologies more understandable, dependable, and open to improvement. We thank the speakers, panelists, session chairs, organizers, and attendees of SSW '25 for the talks and discussions on which this report is based. The workshop program and recordings are available at https://ipads.se.sjtu.edu.cn/sigops-strategic/.

REFERENCES

[1] David H. Abernathy, John S. Mancino, Charls R. Pearson, and Dona C. Swiger. 1973. Survey of design goals for operating systems. *SIGOPS Oper. Syst. Rev.* 7, 3 (July 1973), 19–34. https://doi.org/10.1145/775260.775263

[2] Jeffrey Dean and Luiz André Barroso. 2013. The tail at scale. *Commun. ACM* 56, 2 (Feb. 2013), 74–80. https://doi.org/10.1145/2408776.2408794

[3] Hadi Esmaeilzadeh, Ting Cao, Xi Yang, Stephen M. Blackburn, and Kathryn S. McKinley. 2012. Looking back and looking forward: power, performance, and upheaval. *Commun. ACM* 55, 7 (July 2012), 105–114. https://doi.org/10.1145/2209249.2209272

[4] Butler Lampson. 2000. Computer systems research (invited talk) (summary only): past and future. *SIGOPS Oper. Syst. Rev.* 34, 2 (April 2000), 8–9. https://doi.org/10.1145/346152.346162

[5] Butler Lampson. 2021. Hints and Principles for Computer System Design. arXiv:2011.02455 [cs.DC] https://arxiv.org/abs/2011.02455

[6] Butler W. Lampson. 1983. Hints for computer system design. *SIGOPS Oper. Syst. Rev.* 17, 5 (Oct. 1983), 33–48. https://doi.org/10.1145/773379.806614

[7] C. Mohan. 1978. Survey of recent operating systems research, designs and implementations. *SIGOPS Oper. Syst. Rev.* 12, 1 (Jan. 1978), 53–89. https://doi.org/10.1145/775323.775329

The following authors served as editors of this article:

Haibo Chen (haibochen@sjtu.edu.cn) is a Distinguished Professor in the School of Computer Science at Shanghai Jiao Tong University and Director of the Institute of Parallel and Distributed Systems.

Ding Yuan (yuan@eecg.toronto.edu) is a Professor in the Department of Electrical and Computer Engineering at the University of Toronto.

KyoungSoo Park (kyoungsoo@gmail.com) is a Professor in the Department of Computer Science and Engineering at Seoul National University.

Dong Du (Dd_nirvana@sjtu.edu.cn) is an Associate Professor in the Institute of Parallel and Distributed Systems at Shanghai Jiao Tong University.